

```

From Coq Require Import Arith Lia Reals ZArith Ring Lra.
From Coq Require Import Arith.PeanoNat.

```

```

(* ===== *)
(* Mutual-scale reading of Dedenko's manuscript, version 14. *)
(* Parameters m,p range over the reals; integer and modular *)
(* issues are treated separately. The odd binomial core has *)
(* two real normalisations: core = n!^n and core = 2^(n-1)*q^n. *)
(* Under the mutual scale-symmetry condition l^n = q^n, *)
(* the verified obstruction is n = 2^(n-1), hence n in {1,2}. *)
(* ===== *)

```

```

(* ----- Auxiliary binomial divisibility facts ----- *)

```

```

Local Open Scope Z_scope.

```

```

Lemma Z_sub_split (a b c : Z) : a - b = (a - c) + (c - b).

```

```

Proof. ring. Qed.

```

```

Lemma Zpow_diff_divides (x y : Z) (n : nat) :

```

```

  Z.divide (x - y) (Z.pow x (Z.of_nat n) - Z.pow y (Z.of_nat n)).

```

```

Proof.

```

```

  induction n as [|n IH].

```

```

  - simpl. exists 0%Z; ring.

```

```

  - replace (Z.of_nat (S n)) with (Z.of_nat n + 1) by lia.

```

```

    rewrite !Z.pow_add_r by lia.

```

```

    rewrite !Z.pow_1_r.

```

```

    set (ux := Z.pow x (Z.of_nat n)) in *.

```

```

    set (uy := Z.pow y (Z.of_nat n)) in *.

```

```

    specialize (IH) as [r Hr].

```

```

    rewrite Z.mul_comm with (n := ux) (m := x).

```

```

    rewrite Z.mul_comm with (n := uy) (m := y).

```

```

    rewrite (Z_sub_split (x * ux) (y * uy) (x * uy)).

```

```

    assert (Hx : x * ux - x * uy = x * (ux - uy)) by ring.

```

```

    assert (Hy : x * uy - y * uy = (x - y) * uy) by ring.

```

```

    rewrite Hx, Hy.

```

```

    rewrite Hr.

```

```

    exists (x * r + uy).

```

```

    ring.

```

```

Qed.

```

```

Lemma odd_n_diff_pow_even

```

```

  (a b : Z) (n : nat) :

```

```

  Nat.odd n = true ->

```

```

  Z.divide 2 (Z.pow (a + b) (Z.of_nat n) - Z.pow (a - b) (Z.of_nat n)).

```

```

Proof.

```

```

  intros Hodd.

```

```

  apply Nat.odd_spec in Hodd.

```

```

  destruct Hodd as [k Hk].

```

```

  subst n.

```

```

  replace (Z.of_nat (2 * k + 1)) with (Z.of_nat (S (2 * k))) by lia.

```

```

  specialize (Zpow_diff_divides (a + b) (a - b) (S (2 * k))) as Hdiv.

```

```

  replace ((a + b) - (a - b)) with (2 * b) in Hdiv by ring.

```

```

  destruct Hdiv as [r Hr].

```

```

  exists (b * r).

```

```

  rewrite Hr.

```

```

  ring.

```

Qed.

Close Scope Z_scope.

(* ----- Real-parameter identities (m,p in R) ----- *)

Local Open Scope R_scope.

Lemma sum_diff_from_parameters_R

(n : nat) (m p : R) :

let z := pow m n + pow p n in

let x := pow m n - pow p n in

z + x = 2 * pow m n /\

z - x = 2 * pow p n.

Proof.

intros z x; unfold z, x; split; ring.

Qed.

Close Scope R_scope.

(* ----- Integer-parameter specialisation (m,p in Z) ----- *)

Local Open Scope Z_scope.

Lemma sum_diff_from_parameters_Z

(n : nat) (m p : Z) :

let z := m ^ Z.of_nat n + p ^ Z.of_nat n in

let x := m ^ Z.of_nat n - p ^ Z.of_nat n in

z + x = 2 * m ^ Z.of_nat n /\

z - x = 2 * p ^ Z.of_nat n.

Proof.

intros z x; unfold z, x; split; ring.

Qed.

Corollary parity_condition_Z

(n : nat) (m p : Z) :

let z := m ^ Z.of_nat n + p ^ Z.of_nat n in

let x := m ^ Z.of_nat n - p ^ Z.of_nat n in

Z.even (z + x) = true /\

Z.even (z - x) = true.

Proof.

intros z x.

destruct (sum_diff_from_parameters_Z n m p) as [Hxz Hxz'].

split.

- replace (z + x) with (2 * m ^ Z.of_nat n) by exact Hxz.

rewrite Z.even_mul; simpl; reflexivity.

- replace (z - x) with (2 * p ^ Z.of_nat n) by exact Hxz'.

rewrite Z.even_mul; simpl; reflexivity.

Qed.

Lemma no_parameters_if_parity_violation (n : nat) (z x : Z) :

Z.even (z + x) = false /\ Z.even (z - x) = false ->

~ (exists m p : Z,

z = m ^ Z.of_nat n + p ^ Z.of_nat n /\

x = m ^ Z.of_nat n - p ^ Z.of_nat n).

Proof.

intros Hpar [m [p [Hz Hx]]].

destruct (sum_diff_from_parameters_Z n m p) as [Hsum Hdiff].

```

destruct Hpar as [H1|H2].
- rewrite Hz, Hx, Hsum in H1.
  rewrite Z.even_mul in H1; simpl in H1. discriminate.
- rewrite Hz, Hx, Hdifff in H2.
  rewrite Z.even_mul in H2; simpl in H2. discriminate.
Qed.

```

```

Lemma no_parameters_if_odd (n : nat) (z x : Z) :
  Z.odd (z + x) = true /\ Z.odd (z - x) = true ->
  ~ (exists m p : Z,
    z = m ^ Z.of_nat n + p ^ Z.of_nat n /\
    x = m ^ Z.of_nat n - p ^ Z.of_nat n).

```

```

Proof.
  intros Hodd [m [p [Hz Hx]]].
  destruct (sum_diff_from_parameters_Z n m p) as [Hsum Hdifff].
  destruct Hodd as [H1|H2].
- rewrite Hz, Hx, Hsum in H1.
  rewrite Z.odd_mul in H1; simpl in H1. discriminate.
- rewrite Hz, Hx, Hdifff in H2.
  rewrite Z.odd_mul in H2; simpl in H2. discriminate.
Qed.

```

```

Lemma no_parameters_for_example :
  ~ (exists m p : Z,
    2%Z = m ^ Z.of_nat 3 + p ^ Z.of_nat 3 /\
    1%Z = m ^ Z.of_nat 3 - p ^ Z.of_nat 3).

```

```

Proof.
  apply (no_parameters_if_parity_violation 3 2 1).
  now left.
Qed.

```

Close Scope Z_scope.

(* ----- Checks for the manuscript reconstruction steps ----- *)
 Local Open Scope nat_scope.

```

Lemma equation_rearrange_nat (n x y z : nat) :
  (x ^ n + y ^ n = z ^ n)%nat -> (z ^ n - x ^ n = y ^ n)%nat.

```

```

Proof.
  intro H.
  rewrite <- H.
  lia.
Qed.

```

```

Lemma parameter_equations_raise_nat (n m p z x : nat) :
  z = (m ^ n + p ^ n)%nat ->
  x = (m ^ n - p ^ n)%nat ->
  (z ^ n = (m ^ n + p ^ n) ^ n /\
  x ^ n = (m ^ n - p ^ n) ^ n)%nat.

```

```

Proof.
  intros Hz Hx.
  subst z x.
  split; reflexivity.
Qed.

```

```

Lemma parameterized_difference_substitution_nat (n m p y : nat) :

```

```

    ((m ^ n + p ^ n) ^ n - (m ^ n - p ^ n) ^ n)%nat = y ^ n)%nat ->
    let z := (m ^ n + p ^ n)%nat in
    let x := (m ^ n - p ^ n)%nat in
    (z ^ n - x ^ n = y ^ n)%nat.
Proof.
  intro H.
  exact H.
Qed.

```

Lemma pythagorean_parameter_example :

```

  let m := 3 in let p := 2 in
  let z := (m ^ 2 + p ^ 2)%nat in
  let x := (m ^ 2 - p ^ 2)%nat in
  let y := (2 * m * p)%nat in
  (z ^ 2 = x ^ 2 + y ^ 2)%nat.
Proof.
  simpl.
  reflexivity.
Qed.

```

(* ----- Mutual-scale factorisation, version 14 ----- *)
Local Open Scope R_scope.

Definition real_core_factorization (n : nat) (core l : R) : Prop :=
(0 < n)%nat /\ core = INR n * pow l n.

Definition coefficient_mass_factorization (n : nat) (core q : R) : Prop :=
(0 < n)%nat /\ core = INR (2 ^ (n - 1)) * pow q n.

Definition mutual_scale_symmetry (n : nat) (l q : R) : Prop :=
pow l n = pow q n.

Lemma real_core_factorization_unique_value

```

  (n : nat) (core l1 l2 : R) :
  real_core_factorization n core l1 ->
  real_core_factorization n core l2 ->
  INR n * pow l1 n = INR n * pow l2 n.
Proof.
  intros [_ H1] [_ H2].
  rewrite <- H1, <- H2.
  reflexivity.

```

Qed.

Lemma real_core_factorization_substitutes_y_power

```

  (n : nat) (core l y : R) :
  real_core_factorization n core l ->
  pow y n = 2 * core ->
  pow y n = 2 * INR n * pow l n.

```

Proof.

```

  intros [_ Hcore] Hy.
  rewrite Hy, Hcore.
  ring.

```

Qed.

Lemma coefficient_mass_factorization_substitutes_difference
(n : nat) (core q diff : R) :

```

coefficient_mass_factorization n core q ->
diff = 2 * core ->
diff = INR (2 ^ n) * pow q n.

```

Proof.

```

intros [Hnpos Hcore] Hdiff.
rewrite Hdiff, Hcore.
replace (INR (2 ^ n)) with (2 * INR (2 ^ (n - 1))).
- set (u := pow q n). ring.
- destruct n as [|n].
  + lia.
  + rewrite Nat.pow_succ_r' by lia.
    replace (S n - 1)%nat with n by lia.
    rewrite mult_INR.
    simpl.
    lra.

```

Qed.

Lemma two_scale_factorizations_relation

(n : nat) (core l q : R) :

```

real_core_factorization n core l ->
coefficient_mass_factorization n core q ->
pow l n = pow q n * (INR (2 ^ (n - 1)) / INR n).

```

Proof.

```

intros [Hnpos Hl] [_ Hq].
assert (Heq : INR n * pow l n = INR (2 ^ (n - 1)) * pow q n).
{ rewrite Hl in Hq. exact Hq. }
assert (Hn_pos : 0 < INR n) by (apply lt_0_INR; exact Hnpos).
apply Rmult_eq_reg_l with (r := INR n).
- rewrite Heq. field. lra.
- lra.

```

Qed.

Lemma Rpow_pos_of_pos (a : R) (n : nat) :

0 < a -> 0 < pow a n.

Proof.

```

intro Ha.
induction n as [|n IH]; simpl.
- lra.
- apply Rmult_lt_0_compat; assumption.

```

Qed.

Lemma mutual_scale_symmetry_forces_shift

(n : nat) (core l q : R) :

```

real_core_factorization n core l ->
coefficient_mass_factorization n core q ->
mutual_scale_symmetry n l q ->
0 < q ->
n = (2 ^ (n - 1))%nat.

```

Proof.

```

intros [_ Hl] [_ Hq] Hsym Hqpos.
assert (Heq : INR n * pow q n = INR (2 ^ (n - 1)) * pow q n).
{ pose proof Hl as Hleft.
  unfold mutual_scale_symmetry in Hsym.
  rewrite Hsym in Hleft.
  rewrite Hleft in Hq.
  exact Hq. }

```

```

assert (Hqpow_pos : 0 < pow q n) by now apply Rpow_pos_of_pos.
assert (Hcancel : INR n = INR (2 ^ (n - 1))).
{ apply Rmult_eq_reg_r with (r := pow q n).
  - exact Heq.
  - lra. }
apply INR_eq.
exact Hcancel.
Qed.

Record MutualScaleData : Type := {
  ms_n : nat;
  ms_core : R;
  ms_l : R;
  ms_q : R;
  ms_l_pos : 0 < ms_l;
  ms_q_pos : 0 < ms_q;
  ms_linear_factorization : real_core_factorization ms_n ms_core ms_l;
  ms_coefficient_factorization : coefficient_mass_factorization ms_n
ms_core ms_q;
  ms_mutual_symmetry : mutual_scale_symmetry ms_n ms_l ms_q
}.

```

```

Lemma mutual_scale_data_forces_shift (s : MutualScaleData) :
  ms_n s = (2 ^ (ms_n s - 1))%nat.
Proof.

```

```

  eapply mutual_scale_symmetry_forces_shift.
  - exact (ms_linear_factorization s).
  - exact (ms_coefficient_factorization s).
  - exact (ms_mutual_symmetry s).
  - exact (ms_q_pos s).

```

Qed.

Close Scope R_scope.

(* ----- Modular remark: congruence is weaker than equality -----
- *)

Section ModularRemark.

Local Open Scope Z_scope.

```

Definition modular_congruent (modq a b : Z) : Prop :=
  exists t : Z, a = b + modq * t.

```

```

Lemma integer_equality_implies_modular_power_congruence
  (n : nat) (x y z modq : Z) :
  Z.pow x (Z.of_nat n) + Z.pow y (Z.of_nat n) =
  Z.pow z (Z.of_nat n) ->
  modular_congruent modq
    (Z.pow z (Z.of_nat n))
    (Z.pow x (Z.of_nat n) + Z.pow y (Z.of_nat n)).

```

Proof.

```

  intro H.
  exists 0%Z.
  rewrite <- H.
  ring.

```

Qed.

```

Lemma modular_congruence_has_integer_witness
  (modq lhs rhs : Z) :
  modular_congruent modq lhs rhs -> exists t : Z, lhs = rhs + modq * t.
Proof.
  exact (fun H => H).
Qed.

```

```

Lemma integer_equality_is_zero_modular_witness
  (n : nat) (x y z modq : Z) :
  Z.pow x (Z.of_nat n) + Z.pow y (Z.of_nat n) =
  Z.pow z (Z.of_nat n) ->
  exists t : Z,
  t = 0%Z /\
  Z.pow z (Z.of_nat n) =
  Z.pow x (Z.of_nat n) + Z.pow y (Z.of_nat n) + modq * t.
Proof.
  intro H.
  exists 0%Z.
  split; [reflexivity|].
  rewrite <- H.
  ring.
Qed.

```

```

Lemma modular_congruence_not_integer_equality :
  modular_congruent 5 (Z.pow 3 3) (Z.pow 1 3 + Z.pow 1 3) /\
  Z.pow 1 3 + Z.pow 1 3 <> Z.pow 3 3.
Proof.
  split.
  - exists 5%Z.
    reflexivity.
  - discriminate.
Qed.

```

```

Record ModularParameterResidues : Type := {
  mpr_modq : Z;
  mpr_n : nat;
  mpr_m : Z;
  mpr_p : Z;
  mpr_A : Z;
  mpr_B : Z;
  mpr_z : Z;
  mpr_x : Z;
  mpr_mn_residue :
    modular_congruent mpr_modq (Z.pow mpr_m (Z.of_nat mpr_n)) mpr_A;
  mpr_pn_residue :
    modular_congruent mpr_modq (Z.pow mpr_p (Z.of_nat mpr_n)) mpr_B;
  mpr_z_residue : modular_congruent mpr_modq mpr_z (mpr_A + mpr_B);
  mpr_x_residue : modular_congruent mpr_modq mpr_x (mpr_A - mpr_B)
}.

```

End ModularRemark.

```

(* ----- Elementary growth facts on naturals ----- *)
Local Open Scope nat_scope.

```

Lemma pow2_gt_linear_shift (k : nat) :

$2^{k+3} > 2 * (k+3)$.

Proof.

induction k as [|k IH]; simpl.

- lia.

- replace (S k + 3) with (k + 4) by lia.

replace (2 ^ (S k + 3)) with (2 * 2 ^ (k + 3)) by

(replace (S k + 3) with (S (k + 3)) by lia;

rewrite Nat.pow_succ_r by lia;

rewrite Nat.mul_comm; reflexivity).

assert (IH' : 2 ^ (k + 3) > 2 * (k + 3)) by exact IH.

assert (Htmp : 2 * 2 ^ (k + 3) > 2 * (2 * (k + 3))).

{ apply Nat.mul_lt_mono_pos_l; [lia|exact IH']. }

replace (2 * (2 * (k + 3))) with (4 * (k + 3)) in Htmp by lia.

apply Nat.le_lt_trans with (m := 4 * (k + 3)).

+ lia.

+ exact Htmp.

Qed.

Lemma pow2_gt_linear (n : nat) :

$3 \leq n \rightarrow 2^n > 2 * n$.

Proof.

intros Hn.

destruct (Nat.le_exists_sub 3 n Hn) as [k [Hk _]].

rewrite Hk.

replace (3 + k) with (k + 3) by lia.

apply pow2_gt_linear_shift.

Qed.

Lemma pow_eq_linear_cases (n : nat) :

$2^n = 2 * n \rightarrow n = 0 \vee n = 1 \vee n = 2$.

Proof.

destruct n as [|n].

- simpl. intro H. now left.

- destruct n as [|n].

+ simpl. intro H. right; left; lia.

+ destruct n as [|n].

* simpl. intro H. right; right; lia.

* intro H.

assert (3 <= S (S (S n))) by lia.

specialize (pow2_gt_linear _ H0) as Hgt.

rewrite H in Hgt. lia.

Qed.

Lemma pow_eq_linear_positive (n : nat) :

$2^n = 2 * n \rightarrow n = 1 \vee n = 2$.

Proof.

intro H.

destruct (pow_eq_linear_cases n H) as [H0 | [H1 | H2]].

- subst n. discriminate.

- now left.

- now right.

Qed.

Lemma two_pow_eq_two_mul_iff_shift (n : nat) :

$(2^n = 2 * n) \% \text{nat} \leftrightarrow (n = 2^{(n-1)}) \% \text{nat}$.

Proof.

```
destruct n as [|n].
- simpl. split; intro H; lia.
- split.
  + intro H.
    rewrite Nat.pow_succ_r' in H by lia.
    apply Nat.mul_cancel_l in H; [|lia].
    replace (S n - 1)%nat with n by lia.
    symmetry.
    exact H.
  + intro H.
    rewrite Nat.pow_succ_r' by lia.
    replace (S n - 1)%nat with n in H by lia.
    rewrite H.
    reflexivity.
```

Qed.

Lemma binary_scaling_roots_only_one_two (n : nat) :
(n = 2 ^ (n - 1))%nat -> n = 1%nat \/ n = 2%nat.

Proof.

```
intro Hshift.
apply two_pow_eq_two_mul_iff_shift in Hshift.
now apply pow_eq_linear_positive in Hshift.
```

Qed.

Lemma mutual_scale_data_forces_small_exponent (s : MutualScaleData) :
ms_n s = 1%nat \/ ms_n s = 2%nat.

Proof.

```
apply binary_scaling_roots_only_one_two.
apply mutual_scale_data_forces_shift.
```

Qed.

Theorem mutual_scale_data_excludes_high_exponent (s : MutualScaleData) :
(2 < ms_n s)%nat -> False.

Proof.

```
intro Hgt.
destruct (mutual_scale_data_forces_small_exponent s) as [H1|H2]; lia.
```

Qed.

Goal forall s : MutualScaleData, ms_n s = 1%nat \/ ms_n s = 2%nat.

Proof.

```
exact mutual_scale_data_forces_small_exponent.
```

Qed.

Calculation time: 2.5s

FLT on □ main [!?] ...

→ coqc GlobalNormalization.v

FLT on □ main [!?] took 2.5s ...

→